

AI Customer Support Platform - Project Documentation

1. Abstract

The AI Customer Support Platform is a modern, real-time ticketing system designed to bridge the gap between frustrated customers and overwhelmed support agents. By integrating advanced Large Language Models (LLMs) and Retrieval-Augmented Generation (RAG) techniques, the platform automates initial responses to routine queries while intelligently escalating complex issues to human agents. Support agents are empowered with an "AI Copilot" that analyzes sentiment, summarizes long chat histories, and drafts context-aware replies. The result is a highly responsive, scalable, and intelligent support ecosystem that drastically reduces resolution times and improves customer satisfaction.

2. Problem Statement

Traditional customer support systems suffer from two major bottlenecks:

- Customer Frustration:** Users face long wait times in queues for answers to basic, repetitive questions.
- Agent Burnout:** Human agents are overwhelmed by high volumes of mundane tickets, leaving them with less time to solve complex, nuanced problems that actually require human empathy and critical thinking.

The Goal: To build a real-time ticketing platform where AI instantly resolves common queries and acts as an intelligent assistant for human agents, enabling them to work 10x faster.

3. Technology Stack

Our platform utilizes a cutting-edge, highly responsive technology stack:

- Frontend:** React 19, Vite, Tailwind CSS v4, Framer Motion (for animations), React Router, and Axios.
- Backend:** FastAPI (Python), Uvicorn, SQLAlchemy (SQLite/PostgreSQL).
- Real-time Communication:** WebSockets for instant, bi-directional messaging.
- AI Engine:** Google Gemini API (Deep Learning/Transformers) for Natural Language Processing (NLP), Summarization, and Sentiment Analysis.
- RAG & Memory:** ChromaDB (Vector Database) and Sentence Transformers for semantic search over company knowledge bases.

4. System Architecture

The following diagram illustrates how the different components of the system interact with each other, highlighting the integration of WebSockets for real-time chat and the AI pipeline.

```
architecture-beta
group frontend(cloud) [Frontend App]
group backend(server) [Backend Services]
group ai(cloud) [AI & Database]

service react(server) [React/Vite] in frontend
service fastapi(server) [FastAPI Server] in backend
service ws(server) [WebSocket Manager] in backend

service db(database) [SQL Database] in ai
service chroma(database) [ChromaDB Vector] in ai
service gemini(cloud) [Google Gemini API] in ai

react:R --> L:fastapi
react:R <-- L:ws

fastapi:R --> L:db
fastapi:R --> L:chroma
fastapi:B --> T:ws

chroma:R --> L:gemini
fastapi:R --> L:gemini
```

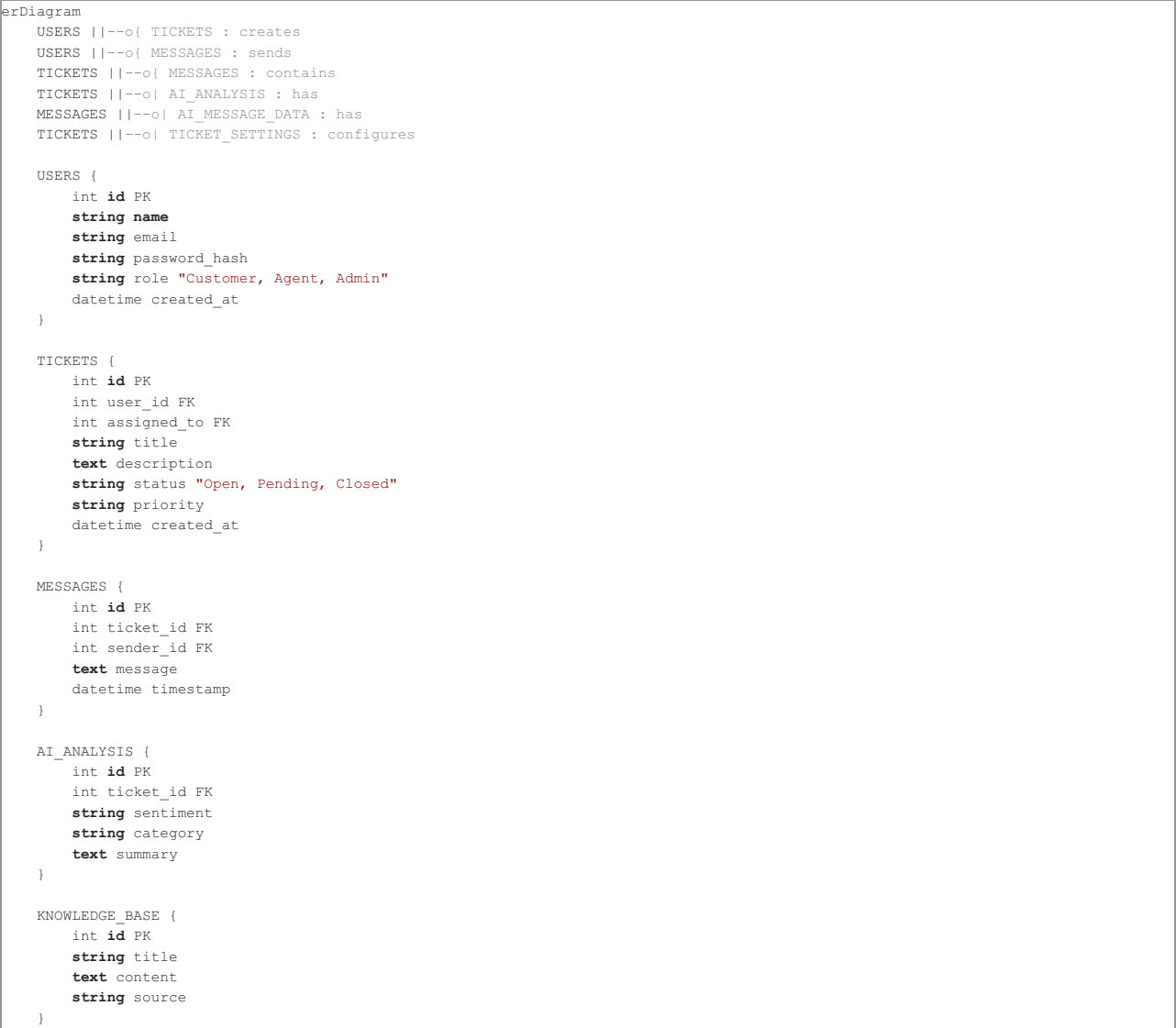
5. System Flowchart

Here is the operational flow when a customer creates a ticket and sends a message.

```
graph TD
    A[Customer Opens Ticket] --> B{AI Auto-Reply Enabled?}
    B -- Yes --> C[Search ChromaDB Knowledge Base]
    C --> D[Retrieve Relevant Context]
    D --> E[Send Context to Gemini AI]
    E --> F[Generate & Send Instant AI Reply via WebSockets]
    B -- No --> G[Notify Human Agent]
    F --> H{Issue Resolved?}
    H -- Yes --> I[Close Ticket]
    H -- No --> G
    G --> J[Agent Opens Dashboard]
    J --> K[AI Copilot Summarizes Chat]
    K --> L[AI Suggests Replies based on Sentiment]
    L --> M[Agent Sends Final Reply]
```

6. Database Schema (ER Diagram)

The relational database manages users, tickets, messages, and AI analysis data.



7. User Interfaces

(Note: Please insert your actual screenshots below each heading)

7.1 Sign In Page

[Insert Screenshot of Sign In Page Here]

Explanation: A secure, modern login portal that routes users to their specific dashboards based on their Role-Based Access Control (RBAC) profile (Customer, Agent, or Admin).

7.2 Customer Ticket Box

[Insert Screenshot of Customer Ticket Box Here]

Explanation: A live chat interface using WebSockets. Customers receive instantaneous replies generated by the AI without needing to refresh the page.

7.3 Agent Dashboard

[Insert Screenshot of Agent Dashboard Here]

Explanation: The control center for support staff. It features the "AI Copilot" sidebar which automatically summarizes long chat histories and detects customer sentiment.

7.4 Admin Dashboard

[Insert Screenshot of Admin Dashboard Here]

Explanation: A high-level overview of system metrics, ticket volumes, and agent performance, providing insights into overall customer satisfaction.

7.5 Knowledge Base

[Insert Screenshot of Knowledge Base Here]

Explanation: The interface where admins upload company rulebooks. This data is converted into vectors and stored in ChromaDB to power the RAG pipeline.

8. API Endpoints Overview

The backend exposes a comprehensive RESTful API alongside WebSocket connections.

(Note: Please insert a screenshot of your Swagger UI / */docs* page here)

[Insert Screenshot of API Endpoints Here]

Key Endpoints:

- **Authentication:**
 - `POST /login` - Generate JWT tokens.
 - `POST /register` - Register a new user.
- **Tickets & Chat:**
 - `GET /tickets` & `POST /tickets` - Manage ticket lifecycle.
 - `GET /messages/{ticket_id}` - Retrieve chat history.
 - `WS /ws/chat/{ticket_id}` - WebSocket endpoint for real-time messaging.
- **AI Integrations:**
 - `POST /ai/summarize/{ticket_id}` - Generate a 3-bullet summary of a chat.
 - `POST /ai/analyze-sentiment/{ticket_id}` - Detect customer emotions.
 - `POST /ai/reply-suggestions/{ticket_id}` - Generate context-aware reply drafts.
- **Knowledge Base (RAG):**
 - `POST /kb` & `GET /kb` - Manage company documents for semantic search.